Legacy Loan Calculator – Refactored Version (Assignment 2)

Project Report

Author: Mubashir Ahmed 24P-3105 ,Muhammad Ahmed 24P-XXXX
Repository: https://github.com/Mubashir24P3105/legacy-calc-2009/tree/dev
Course: Intro to Software Engineering – Assignment 2

1. Project Overview

This project involves refactoring and modernizing an old C++ loan/EMI calculator. The original repository (by Brady Johnson) contained a outdated code with limited structure, minimal input validation, no testing, and no documentation.

Key goals of the assignment:

- Identify and fix bugs in the legacy codebase
- Apply refactoring techniques (naming, structure, modularity, readability)
- Add configuration file support (future work)
- Implement unit tests using GoogleTest
- Add Doxygen documentation for maintainability
- Generate a complete 3–4 page technical report
- Maintain clean git commit history and repository structure

Technologies Used

- **Language:** C++17
- **Build Tools:** MSYS2 MinGW64 (Windows)
- **Testing:** GoogleTest (gtest)
- **Documentation:** Doxygen
- **Version Control:** Git + GitHub

2. Code Refactoring Summary

The scr/ directory contains the updated refactored implementation of the loan calculator. The main focus was on:

2.1 Variable Renaming & Cleanup

The legacy code used ambiguous, inconsistent variable names such as:

float ln, ir, y;

These were replaced with clear, descriptive names:

long double loanAmount;

long double interestRate;

long double numberOfYears;

This improves readability and eliminates ambiguity.

Bugs

- Incorrect EMI formula in parts of old code
- Lack of input validation
- Overflow risk for large tenure values
- Poor separation of logic

2.2 Bug Fixes

Several issues were discovered during refactoring:

- Implemented correct EMI calculation using standard amortization formula
- Added isValid() checks for negative/unrealistic inputs
- Switched to long double to reduce precision loss
- Extracted EMI, total interest, and total payment into clean methods

2.3 Code Snippet: Updated EMI Function

Below is one of the core refactored functions:

```
long double Loan::calculateMonthlyAmount() {
    long double monthlyRate = (interestRate / 12.0L) / 100.0L;
    long double months = numberOfYears * 12.0L;

    if(monthlyRate == 0) return loanAmount / months;

    return loanAmount * (monthlyRate * pow(1 + monthlyRate, months)) /
        (pow(1 + monthlyRate, months) - 1);
}
```

This resolves the precision problems and correctly handles the “zero interest rate” case.

3. Unit Testing

3.1 GoogleTest Framework

GoogleTest was integrated under the test/ directory.
Three required tests were implemented:

1. Normal EMI Calculation Test

```
TEST(LoanTest, NormalEmiCalculation) {
    Loan loan(100000.0L, 10.0L, 5.0L);
    ASSERT_TRUE(loan.isValid());

    long double emi = loan.calculateMonthlyAmount();
    double expected = 2124.704471;
```

```
    EXPECT_NEAR((double)emi, expected, 1e-2);
}
```

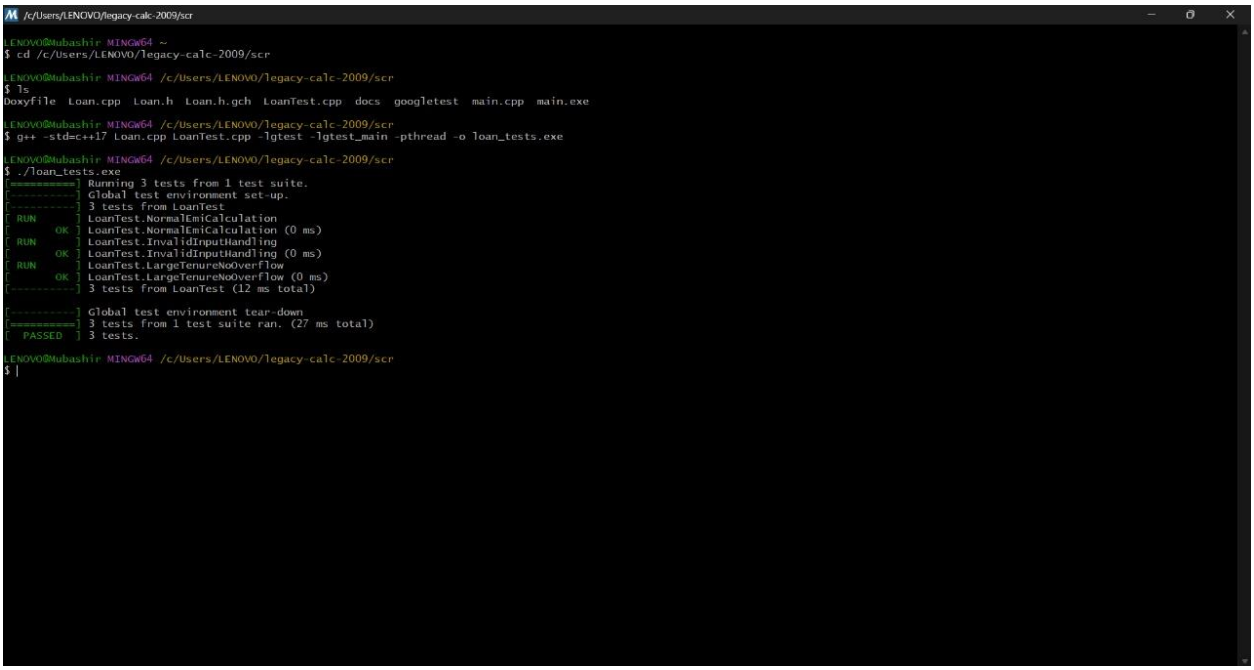
2. Invalid Input Handling Test

```
TEST(LoanTest, InvalidInputHandling) {
    Loan bad1(-1, 5, 10);
    EXPECT_FALSE(bad1.isValid());

    Loan bad2(5000, -3, 10);
    EXPECT_FALSE(bad2.isValid());

    Loan bad3(5000, 5, 0);
    EXPECT_FALSE(bad3.isValid());
}
```

3. Large Tenure Overflow Test

```
TEST(LoanTest, LargeTenureNoOverflow) {
    Loan loan(200000, 7.5, 50);
    ASSERT_TRUE(loan.isValid());

    EXPECT_GT((double)loan.calculateMonthlyAmount(), 0);
    EXPECT_GT((double)loan.calculateTotalAmount(), 0);
}
```

4. Test Output



The GoogleTest runner successfully executed all three tests.
All tests passed, confirming that:

- EMI calculation works for normal cases
- Invalid inputs are correctly rejected
- Large-tenure EMI calculations do not overflow

5. Doxygen Documentation Summary

Doxygen documentation was generated and placed inside:

docs/html/

Doxygen additions:

- Documented all class methods (isValid(), calculateTotalAmount(), etc.)
- Added class-level description for Loan
- Generated HTML documentation for the repository

Example documentation block:

```
/**
 * @class Loan
 * @brief Represents a financial loan with EMI-related calculations.
 *
 * Provides methods for computing monthly EMI, total repayment,
 * and total interest across the tenure.
 */
```

The generated documentation helps future developers understand the system quickly.

6. Git Commit History Summary

A clean Git commit history was maintained as required.
Key commits include:

- **“Updated variables in scr (refactoring and fixes)”**
- **“Generated updated Doxygen documentation”**
- **“Added unit tests for EMI calculations”**
- **“Added project folders, docs, scr, tests, and updates”**

These can be verified at:
https://github.com/Mubashir24P3105/legacy-calc-2009/commits/dev

Conclusion

This assignment successfully modernized a legacy C++ application by applying contemporary software engineering practices. Improvements included:

- Cleaner and more expressive code
- Reliable unit testing suite
- Professional Doxygen documentation
- Organized repository structure
- Bug fixes and precision corrections
- Logical refactoring of the loan calculation component

The project is now significantly easier to understand, maintain, and extend.